Experiment No.: 5

1. **Aim:**. Design and implement RNN for classification of temporal data

2. **Objectives: From this experiment, the student will be able to**

   Learn different Deep Neural Networks.

3. **Outcomes:** Learner will be able to

   CO5: Identify the importance of RNN for sequence learning.

4. **Software Required:** Python

5. **Theory:**

Recurrent Neural Networks (RNNs) are a class of deep learning models specifically designed for sequential or temporal data. Unlike traditional feedforward neural networks, RNNs have memory elements that allow them to retain information from previous time steps, making them ideal for applications such as time-series prediction, speech recognition**,** and natural language processing (NLP). In this experiment, we will design and implement an RNN for classifying sequential data, using a dataset such as IMDB movie reviews (sentiment classification) or sensor data classification**.**

6. **Design:**

a) Problem Definition

- Define the objective of the RNN model.
- Example: Classify IMDB movie reviews as positive or negative.
- Identify the type of sequential data (e.g., text, time-series, stock prices, or sensor data).

b. Data Collection and Preprocessing

- Choose a suitable data set (IMDB, Sensor data, Speech data, etc.).
- Convert the data into a numerical format if it's in text form (Tokenization).
- Normalize or scale numerical data (if required).
- Pad sequences to make them uniform length

c. Model Architecture Design

a) Choose an appropriate RNN architecture:
- Simple RNN (for short sequences)
- LSTM or GRU (for long-term dependencies)

b) Define layers:
- Embedding Layer (for text data)
- RNN Layer (SimpleRNN, LSTM, or GRU)
- Dense Layer (for classification output)

c) Select activation functions:

- ReLU, Sigmoid, Softmax based on classification type.

d. Model Compilation

- Choose an optimizer (e.g., Adam, RMSprop).
- Select a loss function based on the classification task:
    - Binary classification → Binary Cross entropy
    - Multi-class classification → Categorical Cross entropy
- Define evaluation metrics (accuracy, precision, recall, etc.).

e. Model Training

- Select batch size and number of epochs.
- Use validation data to monitor performance.
- Apply early stopping if overfitting occurs.

f. Model Evaluation

- Evaluate performance on test data.
- Analyze accuracy, confusion matrix, F1-score, and loss curves.

g. Performance Tuning & Improvements

- If overfitting occurs:
    - Use dropout layers or L2 regularization.
    - Try LSTM or GRU instead of Simple RNN.
- If accuracy is low:
    - Adjust learning rate, batch size, or network depth.
- Perform hyperparameter tuning.

7. Procedure Steps:

a) Import Required Libraries

b) Load and Preprocess Data

c) Build the RNN Model

d) Train the Model

e) Evaluate the Model

f) Results.

8. Conclusion/ Results:

RNNs are effective for sequence classification tasks as they can process sequential data by maintaining dependencies between past and present inputs. Using embedding layers helps in processing text data by converting words into dense vector representations, making it easier for the model to learn patterns. While Simple RNN can capture short-term dependencies, it struggles with long-term dependencies due to the vanishing gradient problem. To overcome this, LSTM and GRU are recommended as they have specialized gating mechanisms that help retain long-term dependencies, improving the performance of sequence-based tasks like text classification, speech recognition, and time-series forecasting.

9. Viva Question:

- Why are RNNs effective for sequence classification tasks?
- What is the purpose of using an embedding layer in text processing?
- How do Simple RNN handle short-term dependencies?
- What challenges do Simple RNN face when dealing with long-term dependencies?
- Why are LSTM and GRU preferred over Simple RNN for long-term dependencies?

10. References:

1. Francois Chollet, “Deep learning with Python” – Manning Publications.
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville. ―Deep Learning‖, MIT Press Ltd, 2016.
3. https://www.tensorflow.org/guide/keras/rnn
4. https://www.manning.com/books/deep-learning-with-python.
5. http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture10.pdf